

Part 1:

Simple Kafka Producer client API với Java

set up Kafka cluster với 3 brokers trên môi trường local để practice với Producer và hiểu về Kafka storage architecture.

1) Start Kafka cluster

Mở terminal và cd đến Kafka dir, bắt đầu cuộc hành trình. Cd đến config/ và clone **server.properties** ra 2 file đặt tên lần lượt là **server-1.properties**, **server-2.properties**.

```
$ cp config/server.properties config/server-1.properties && \
  cp config/server.properties config/server-2.properties
```

Với prod env, mỗi broker là một physical machine/virtual machine/container nên không cần sửa các config liên quan đến port và data dir. Tuy nhiên chúng ta chạy multi brokers trên cùng host nên cần có chút thay đổi.

Với **server-1.properties**, tìm config key và sửa như sau:

```
broker.id=1

listeners=PLAINTEXT://:9093

log.dirs=/home/admin/kafka_2.13-2.8.0/data/kafka-1
```

File **server-2.properties**:

```
broker.id=2
```

```
listeners=PLAINTEXT://:9094
```

```
log.dirs=/home/admin/kafka_2.13-2.8.0/data/kafka-2
```

Start 2 Kafka broker với 2 terminal mới để join vào cụm [Kafka cluster sẵn có](#).

```
$ kafka-server-start.sh config/server-1.properties
```

```
$ kafka-server-start.sh config/server-2.properties
```

Kiểm tra với câu lệnh:

```
$ zookeeper-shell.sh localhost:2181 ls /brokers/ids
```

```
Connecting to localhost:2181
```

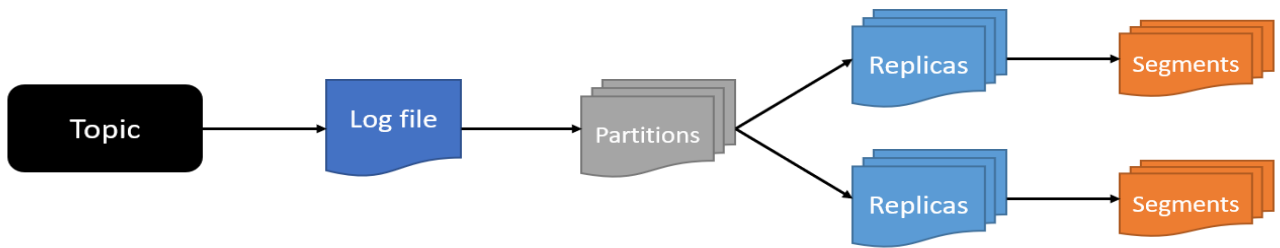
```
WATCHER::
```

```
WatchedEvent state:SyncConnected type:None path:null
```

```
[0, 1, 2]
```

Perfect, Kafka cluster đã ready với 3 broker ids: 0, 1, 2.

2) Kafka storage architecture



Kafka tạo **log file** cho topic để lưu trữ message. **Log file** này được partition, replicated và segmented thành nhiều phần khác nhau.

Tạo topic mới với 5 partition và replication-factor = 3:

```
$ kafka-topics.sh \
  --bootstrap-server localhost:9092,localhost:9093 \
  --partitions 5 \
  --replication-factor 3 \
  --topic new-kafka-topic \
  --create
```

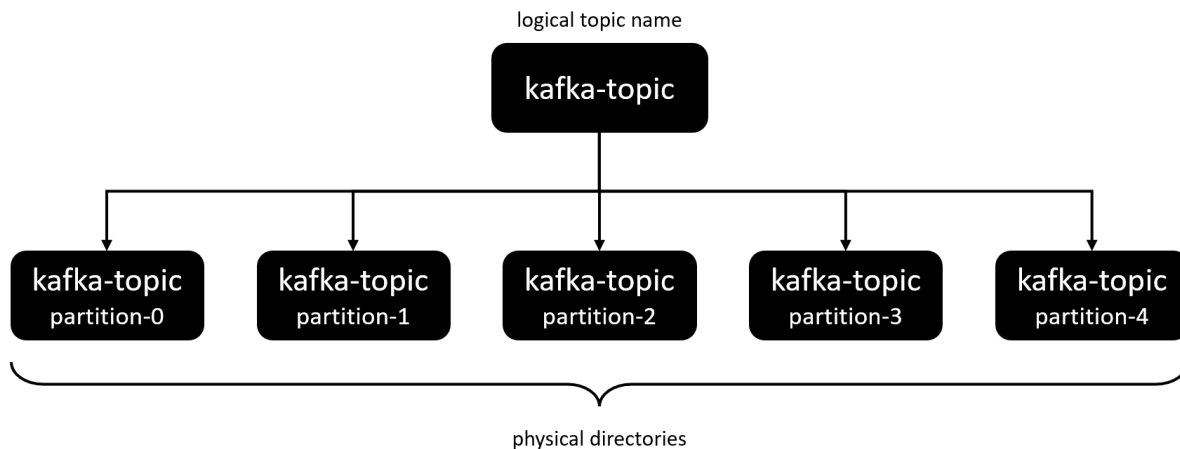
Check data dir của broker bất kì sẽ thấy 5 directories tương ứng với 5 partitions của **new-kafka-topic** được tạo ra:

```
$ ls -d data/kafka-0/* | grep new-kafka-topic

data/kafka-0/new-kafka-topic-0
data/kafka-0/new-kafka-topic-1
data/kafka-0/new-kafka-topic-2
data/kafka-0/new-kafka-topic-3
data/kafka-0/new-kafka-topic-4
```

Check data dir của các broker còn lại cũng nhận kết quả tương tự. Vì chúng ta đang để thông số replication-factor = 3, có nghĩa là 1 partition được replicate thành 3 phiên bản ở 3 broker khác nhau.

Tạo topic mới **other-kafka-topic** với replication-factor = 2, mỗi partition chỉ có 2 directories nằm rải rác ở broker khác nhau.



Mỗi partition được lưu trữ tại một directory:

- Tạo topic với 3 partitions được lưu trữ tương ứng trên 3 directories.
- Topic với 10 partitions là 10 directories.

Ngoài ra còn hệ số **replication-factor**, như vậy tổng số lượng replicas của một topic là **replication-factor * number of partitions**. Với ví dụ trên **new-kafka-topic** có tổng cộng $3 * 5 = 15$ directories để lưu trữ message.

Dĩ nhiên message không được lưu ở directory mà được lưu ở file nằm trong directory. Các file đó dưới dạng extension .log được gọi là **log segment**.

Kiểm tra với cmd:

```
$ ls data/kafka-0/new-kafka-topic-0/ | grep log
```

```
00000000000000000000.log
```

Hiện tại chưa có message nào nên chỉ thấy một file. Khi số lượng message đạt đến mức nhất định, Kafka tạo một file khác để lưu trữ. Default mỗi **log segment** có size là 1 GB.

Log segment file name được đặt tên với message offset đầu tiên của mỗi segment:

```
00000000000000000000.log
000000000000000092773.log
00000000000000002696461.log
```

Đó là toàn bộ cách Kafka lưu trữ message trong mỗi broker với **directory** và **file**. Tất cả đều clear và dễ hiểu, không có gì bí thuật ở đây.

3) Producer

Tạo Java project với Maven hoặc Gradle tùy thuộc sở thích của bạn.

Thêm dependencies sau nếu sử dụng Gradle:

```
dependencies {
    implementation 'org.apache.kafka:kafka-clients:2.8.0'
    implementation 'org.slf4j:slf4j-simple:1.7.32'
}
```

Với Maven:

```
<dependencies>
  <dependency>
    <groupId>org.apache.kafka</groupId>
```

```
<artifactId>kafka-clients</artifactId>

<version>2.8.0</version>

</dependency>

<dependency>

    <groupId>org.slf4j</groupId>

    <artifactId>slf4j-simple</artifactId>

    <version>1.7.32</version>

</dependency>

</dependencies>
```

Tạo Producer với các **properties**:

```
final var props = new Properties();

props.setProperty(ProducerConfig.CLIENT_ID_CONFIG, "java-producer");

props.setProperty(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
"localhost:9092,localhost:9093");

props.setProperty(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());

props.setProperty(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());
```

Có 3 **properties** cần quan tâm:

- **CLIENT_ID_CONFIG**: optional config, nhằm mục đích tracking nguồn produce message.
- **BOOTSTRAP_SERVERS_CONFIG**: địa chỉ đích của Kafka cluster để init connection. Sau khi connect thành công, Kafka producer tự động query metadata và connect đến toàn bộ broker trong mạng Kafka cluster. Có thể sử dụng 1, 2 hoặc tất cả broker address theo format **broker-1-host:broker-1-port,broker-2-host:broker-2-port**. Best practice là

define 2 - 3 address để trong trường hợp bất kì broker nào không connect được, Kafka producer sẽ connect đến broker khác.

- **KEY_SERIALIZER_CLASS_CONFIG** và **VALUE_SERIALIZER_CLASS_CONFIG**: tất cả message gửi đến Kafka bao gồm key và value. Key có thể null nhưng message gửi đi luôn bao gồm cặp key value. Message gửi đi trong network là byte array, do vậy data cần được serialized thành bytes. Kafka producer API cung cấp sẵn vài serializer thông dụng và chúng ta chỉ cần dùng nó. Nhưng trong thực tế, thường sử dụng custom serializer với JSON bằng cách implement `Serializer<T>` interface.

Tiếp theo, tạo Producer và gửi 100 messages đến topic. **ProducerRecord<K, V>** cung cấp nhiều arguments để tạo message trong đó có 2 mandatory fields và 4 optional fields:

- **String topic** (required): topic name.
- **Object message** (required): message cần gửi.
- **Integer partition**: partition muốn gửi message đến.
- **Long timestamp**: timestamp của message với đơn vị millis. Default là `System.currentTimeMillis()`.
- **Object key**: message key. Message cùng key được route đến cùng partition.
- **Iterable<Header> headers**: record header.

```
// Gửi message đến topic và không define key.  
// Message được route đến partition bất kì của topic.  
public ProducerRecord(String topic, V value);  
  
// Gửi message đến topic có define key.  
// Các message cùng key được route đến cùng một partition.  
public ProducerRecord(String topic, K key, V value);  
  
// Gửi message đến chính xác topic partition.
```

```

public ProducerRecord(String topic, Integer partition, K key, V value);
try (var producer = new KafkaProducer<String, String>(props)) {
    for (int i = 0; i < 100; i++) {
        final var message = new ProducerRecord<>(
            "new-kafka-topic",    //topic name
            "key-" + i,          // key
            "message: " + i      // value
        );
        producer.send(message);
    }
}

```

Với đoạn code trên mình sử dụng try-with-resources để producer tự động flush và close resource. Tương đương với:

```

final var producer = new KafkaProducer<String, String>(props);
for (int i = 0; i < 100; i++) {
    final var message = new ProducerRecord<>(
        "new-kafka-topic",    //topic name
        "key-" + i,          // key
        "message: " + i      // value
    );
    producer.send(message);
}
producer.flush();

```



```
producer.close();
```

Phía dưới Kafka client sử dụng buffer và background I/O thread để xử lý. Do đó nếu không flush() thì message chưa được gửi đi ngay hoặc chỉ gửi đi một vài message, không close() có thể dẫn tới resource leak.

Method close() sẽ tự động trigger flush().

Run code thôi, full class như sau:

```
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.kafka.common.serialization.StringSerializer;

import java.util.Properties;

public class Producer {

    public static void main(String[] args) {

        final var props = new Properties();

        props.setProperty(ProducerConfig.CLIENT_ID_CONFIG, "java-
producer");

        props.setProperty(ProducerConfig.BootstrapServers_CONFIG,
"localhost:9092,localhost:9093");

        props.setProperty(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());

        props.setProperty(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());
```

```

try (var producer = new KafkaProducer<String, String>(props)) {
    for (int i = 0; i < 100; i++) {
        final var message = new ProducerRecord<>(
            "new-kafka-topic",    //topic name
            "key-" + i,          // key
            "message: " + i      // value
        );
        producer.send(message);
    }
}
}
}

```

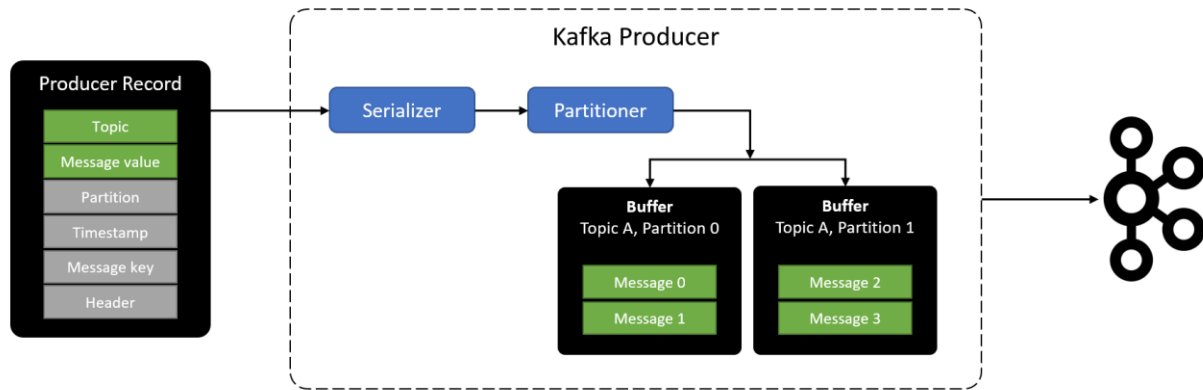
Run consumer CLI để check:

```

$ kafka-console-consumer.sh \
    --bootstrap-server localhost:9092 \
    --topic new-kafka-topic \
    --from-beginning
message: 1
message: 6
message: 7
message: 8
message: 4
message: 5

```

...



4) Serializer

Sau khi gọi `send()` method, message được đưa đến tầng **serializer**. Nhiệm vụ của **serializer** là biến đổi message ở dạng object sang `byte[]` trước khi gửi đến Kafka cluster.

Nếu coi quá trình produce message là việc phân phối đến các quán mát-xa-ge thì **Serializer** sẽ đảm nhận nhiệm vụ đánh bóng, tân trang theo sở thích mà khách mong muốn.

Kafka cung cấp sẵn nhiều Serializer như:

- `StringSerializer`.
- `LongSerializer`.
- `DoubleSerializer`.
- `FloatSerizalizer`....

Chỉ cần khai báo và sử dụng là xong. Trong thực tế các message muốn gửi thường dưới dạng Object, do đó chúng ta có thể tự tạo custom serializer bằng cách implement interface `Serializer<T>`.

Ví dụ code với object serializer:

```

class DriverLocation {

    private long driverId;

    private long longitude;

    private long latitude;

}

class DriverLocationSerializer implements Serializer<DriverLocation> {

    private static final ObjectMapper OBJECT_MAPPER = new ObjectMapper();

    @Override
    public byte[] serialize(final String topic, final DriverLocation data)
    {
        try {
            return objectMapper.writeValueAsString(data).getBytes();
        } catch (Exception e) {
            throw new IllegalArgumentException("Cannot serialize message to
json");
        }
    }
}

```

5) Partitioner

Với Kafka, **partitioner** là một điều phối viên. Internally, Kafka sử dụng **DefaultPartitioner** để route message đến các partition:

- Nếu record được define key mà không define partition, Kafka sẽ hash key và các message cùng key được route đến cùng partition.
- Nếu record được define destination partition, Kafka route message đến chính xác partition đó.
- Nếu record không define destination partition và message key, Kafka route message với round-robin sử dụng **StickyPartitionCache**. Nếu muốn tìm hiểu kĩ hơn có thể đọc code của Kafka nhé.

```
var message = new ProducerRecord<>("topic-name",  
                                     PARTITION,  
                                     "key",  
                                     "message");
```

Nếu không hài lòng với **DefaultPartitioner**, chúng ta có thể customize partitioner bằng cách implement interface **Partitioner**.

```
class CustomPartitioner implement Partitioner {  
    // Implement code here  
}
```

Sau đó add thêm config properties khi khởi tạo Producer:

```
props.put(ProducerConfig.PARTITIONER_CLASS_CONFIG,  
CustomPartitioner.class.getName());
```

6) Message timestamp

ProducerRecord có argument liên quan đến message timestamp và nó là optional. Tuy nhiên trong các hệ thống real-time application, vấn đề liên quan đến thời điểm gửi/nhận message là cực kì quan trọng, do đó nếu không explicit define message timestamp, **Producer** sẽ sử dụng **System.currentTimeMillis()** làm giá trị mặc định.

Có 2 loại message timestamp cần quan tâm:

- **Create time**: thời gian khi message được gửi đi.
- **Log append time**: thời gian khi Kafka nhận được message.

Khi tạo topic chúng ta sẽ define loại message timestamp mong muốn bằng config:

```
message.timestamp.type=CreateTime
```

```
message.timestamp.type=LogAppendTime
```

Default Kafka sử dụng `message.timestamp.type=CreateTime`. Nếu muốn sử dụng **log append time**, cần thay đổi giá trị thành `LogAppendTime`.

```
$ kafka-topics.sh \  
  --bootstrap-server localhost:9092,localhost:9093 \  
  --partitions 5 \  
  --replication-factor 3 \  
  --topic log-append-time \  
  --config message.timestamp.type=LogAppendTime \  
  --create
```

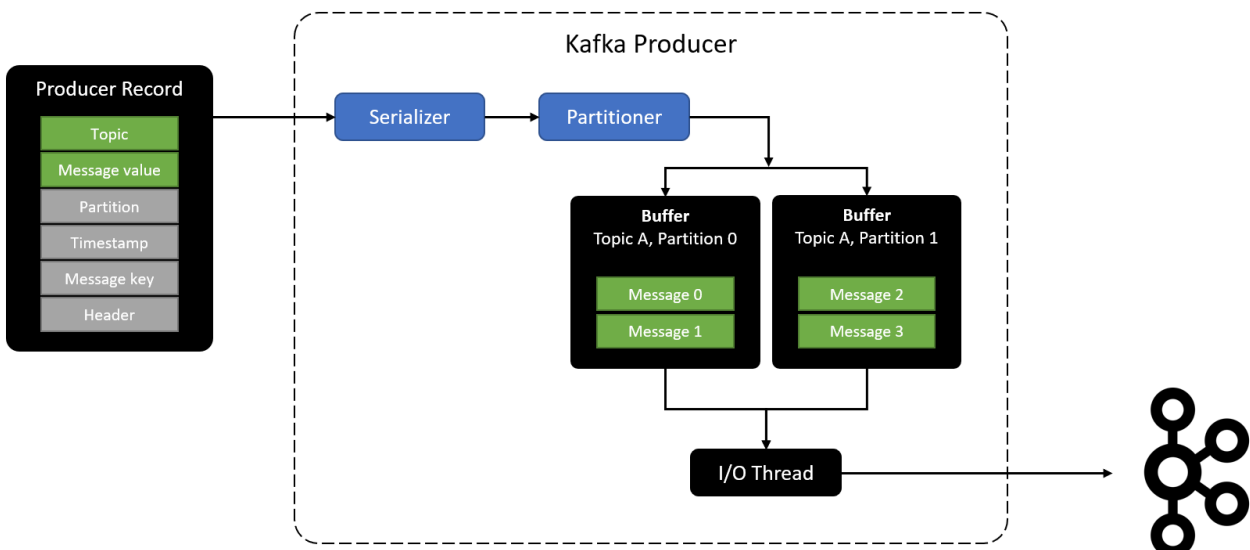
Như vậy dẫn đến 2 trường hợp:

- Sử dụng `message.timestamp.type=CreateTime`, nếu không define argument timestamp, **Producer** sử dụng default value là **`System.currentTimeMillis()`**. Nếu define timestamp, producer sử dụng timestamp làm message timestamp và giá trị được giữ nguyên đến khi message được append vào log.
- Sử dụng `message.timestamp.type=LogAppendTime`, việc define timestamp không còn ý nghĩa. Tất cả message khi đến Kafka sẽ được override timestamp trước khi append log.

7) Message buffer và Producer I/O thread

Sau khi **partitioner** điều phối đến **xe chuyên dụng**, tổ lái không xuất phát ngay mà chờ cho đến khi đạt đủ số lượng nhất định. Tránh trường hợp đi lại nhiều lần, mất thời gian, tốn xăng.

Tương tự với Kafka, các message chưa được gửi đi ngay mà được đẩy vào message buffer để chờ. Background sẽ có I/O thread làm nhiệm vụ gộp các message ở buffer thành request để gửi đến Kafka cluster.



Như vậy, **message buffer** đem đến 2 ưu điểm:

- **Asynchronous:** `send()` method là async và return ngay lập tức, application có thể tiếp tục thực thi mà không bị block.
- **Network roundtrip optimization:** background I/O thread sẽ đóng gói nhiều message lại thành một package và gửi đi trong 1 request, giảm roundtrip đến Kafka cluster, tăng throughput.

Chả có gì là hoàn hảo, nó cũng có nhược điểm to không kém.

Vì muốn tiết kiệm thời gian, xăng dầu và tăng số lượng có thể chở trong một chuyến hàng, các quái xế cố nán lại thêm tí nào hay tí ấy.

- Đen đũi thay số lượng quá đông, các quái xế trở tay không kịp dẫn đến tràn hàng.
- Mọi người bu nhau vào xem , **đội quản lý thị trường** lập tức có mặt để giải quyết thì phát hiện lậu, ập vào thu hồi xử lý. Thế là toi, tưởng ship được nhiều nhưng lại thành không ship được em nào.
- Các quái xế đã rút kinh nghiệm nếu chờ quá 2 phút hoặc số lượng vượt quá 5 thì tự động xuất phát.

Message buffer và I/O thread hoạt động tương tự idea trên, sử dụng các config liên quan đến window-size và window-time để control việc send message đến Kafka.

7.1) Buffer size

Số lượng message gửi đến quá nhiều mà I/O thread xử lý không kịp dẫn đến tràn buffer. Method `send()` sau đó sẽ bị block cho đến khi có free space. Trong tình huống này, việc gửi message mất nhiều thời gian, `send()` method bị block quá lâu sẽ bắn ra `TimeoutException`. Để xử lý vấn đề này chúng ta cần tăng **buffer size** của producer với config `buffer.memory`. Giá trị default của `buffer.memory` là 32 MB.

Thực hiện tăng `buffer.memory` lên 64 MB bằng cách:


```
props.setProperty(ProducerConfig.BUFFER_MEMORY_CONFIG, String.valueOf(64 * 1024 * 1024L));
```

7.2) Batch size

Message đang nằm trong buffer chờ được gửi đi thì bụp.. mất điện. Toàn bộ message bay theo cú chập điện vừa rồi.. và lost message. Tình huống này thì.. trời cứu.

I/O thread dựa trên `batch.size` để process message trong buffer gửi đến Kafka cluster.

Khi nhiều message được gửi đến cùng partition, để tiết kiệm roundtrip, các message được nhóm vào một batch. Khi batch đạt đến size nhất định, I/O thread tạo request và gửi batch đi.

Batch size tỉ lệ thuận với throughput và khả năng lost message. Batch size càng nhỏ khả năng lost message càng giảm kéo theo throughput thấp đi. Giá trị default của batch size là 2048 KB.

Nếu muốn message được gửi đi ngay lập tức, chỉ cần set config batch size = 0. Kéo theo throughput và application performance giảm.

```
props.setProperty(ProducerConfig.BATCH_SIZE_CONFIG, String.valueOf(2048 * 8));
```

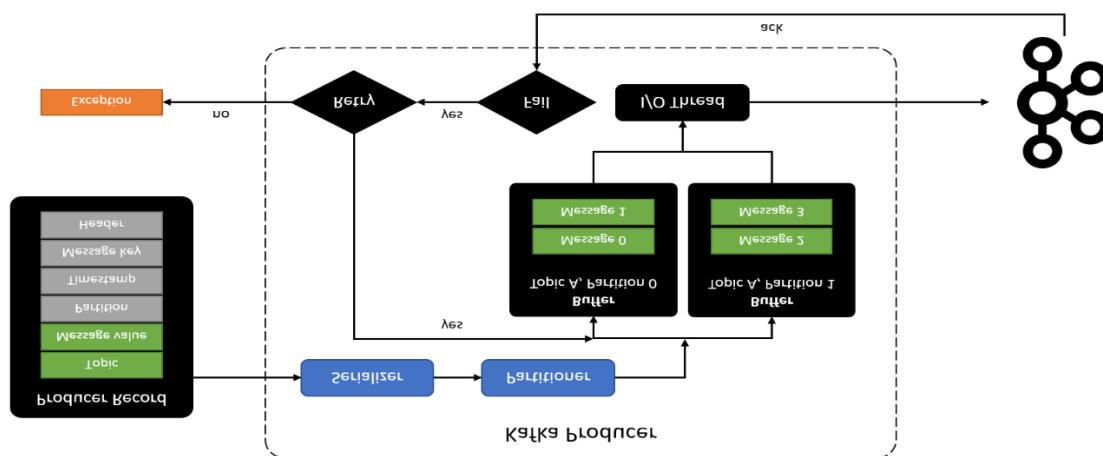
Ngoài ra, sử dụng **flush()** để force I/O thread gửi message đến Kafka.

8) Retry

I/O thread tạo request để gửi message đến Kafka. Mọi chuyện chưa kết thúc ở đây. Cần đảm bảo message đã đến đích và Kafka broker đã append log thành công.

Có 2 trường hợp xảy ra sau khi I/O thread send message:

- Kafka broker sau khi nhận message gửi lại ACK. Nếu append log thành công, Kafka trả về success ack và producer ghi nhận vào hệ thống.
- Nếu append log fail hoặc chờ quá lâu không thấy phản hồi, I/O thread sẽ retry vài lần cho đến khi thành công. Nếu quá số lần retry sẽ throw exception.



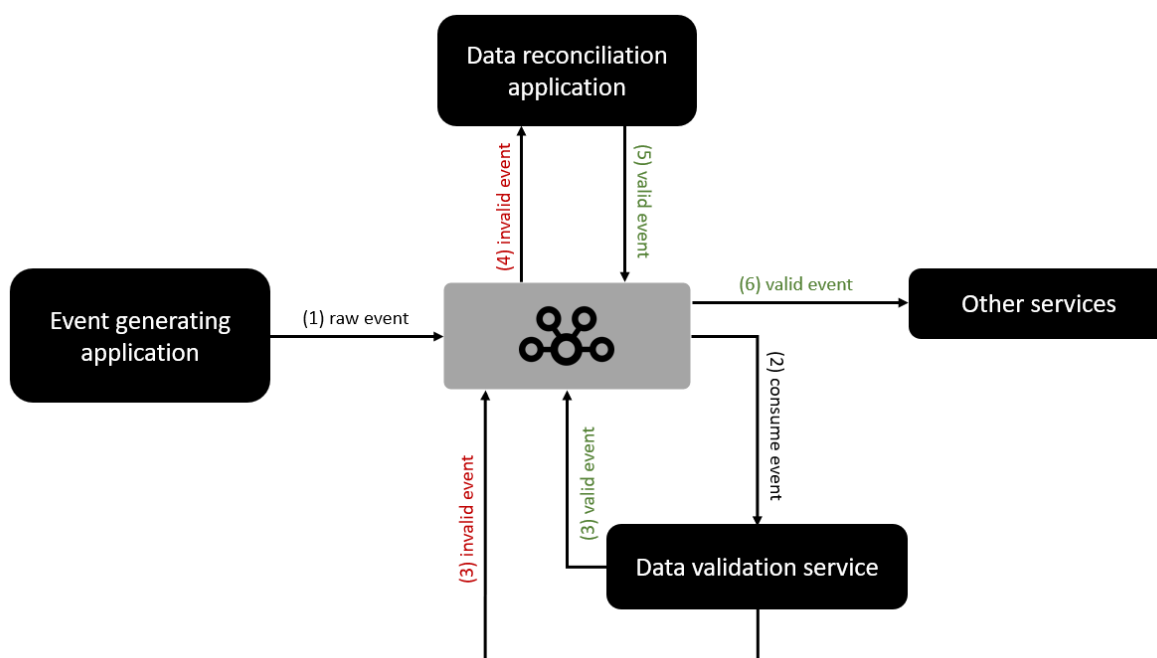
Set up retry times với **Producer** bằng config `retries`.

```
props.setProperty(ProducerConfig.RETRIES_CONFIG, "10");
```

Part 2: Simple Kafka Consumer

1) Scenario

Thực tế, một hệ thống với Kafka có thể hoạt động với mô hình dưới đây:

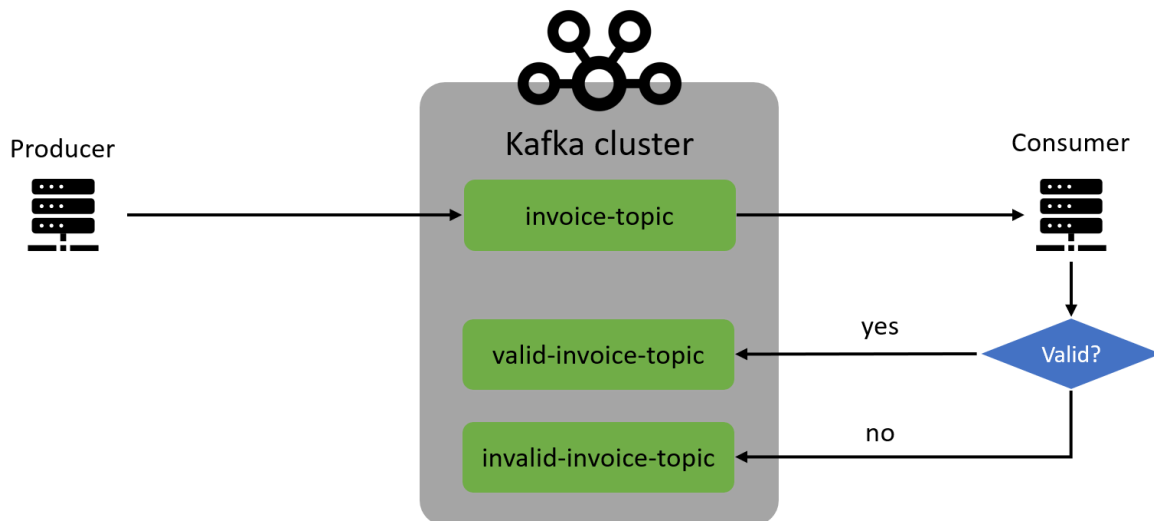


- **Event generating service:** là service produce message tới Kafka.
- **Data validation service:** consume message, thực hiện validate, dựa trên result để produce message tương ứng tới topic.
- **Data reconciliation application:** nếu message invalid, DRA consume message, tiến hành correct message và produce valid message với Kafka.
- **Other service:** cuối cùng message valid được đẩy tới next service để xử lý các business logic phù hợp với yêu cầu.

Với bài toán trên, ta sử dụng **Data validation service** làm ví dụ cho phần consumer:

- Đầu vào là các hóa đơn, producer produce data đến **invoice-topic**.

- Consumer consume message từ **invoice-topic** và xử lý, nếu valid sẽ chuyển tiếp đến **valid-invoice-topic**, ngược lại produce đến **invalid-invoice-topic**.



2) Practice

Có bài toán rồi, quấy thôi. Thêm dependency Kafka client và Jackson để xử lý JSON data:

Với Maven:

```
<dependencies>
  <dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka-clients</artifactId>
    <version>2.8.0</version>
  </dependency>
  <dependency>
    <groupId>org.slf4j</groupId>
```

```

        <artifactId>slf4j-simple</artifactId>

        <version>1.7.32</version>
    </dependency>

    <dependency>

        <groupId>com.fasterxml.jackson.core</groupId>

        <artifactId>jackson-databind</artifactId>

        <version>2.12.5</version>
    </dependency>

    <dependency>

        <groupId>org.projectlombok</groupId>

        <artifactId>lombok</artifactId>

        <version>1.18.20</version>
    </dependency>
</dependencies>

```

Gradle:

```

dependencies {

    implementation 'org.apache.kafka:kafka-clients:2.8.0'

    implementation 'org.slf4j:slf4j-simple:1.7.32'

    implementation 'com.fasterxml.jackson.core:jackson-databind:2.12.5'

}

```

2.1) Custom serializer với JSON:

Với bài toán này, cùng practice tạo một custome serializer/deserializer với JSON type như sau.

Serializer:

```
public class JsonSerializer<T> implements Serializer<T> {

    private final ObjectMapper objectMapper = new ObjectMapper();

    @Override
    public byte[] serialize(String topic, T data) {
        if (data == null) {
            return null;
        }
        try {
            return objectMapper.writeValueAsBytes(data);
        } catch (Exception e) {
            throw new SerializationException("Error serializing JSON
message", e);
        }
    }
}
```

Deserializer:

```
public class JsonDeserializer<T> implements Deserializer<T> {

    private final ObjectMapper objectMapper = new ObjectMapper();
    private Class<T> className;
```

```
    public static final String KEY_CLASS_NAME_CONFIG = "key.class.name";

    public static final String VALUE_CLASS_NAME_CONFIG =
"value.class.name";

    @SuppressWarnings("unchecked")
    @Override

    public void configure(Map<String, ?> props, boolean isKey) {

        if (isKey) {

            className = (Class<T>) props.get(KEY_CLASS_NAME_CONFIG);

            return;

        }

        className = (Class<T>) props.get(VALUE_CLASS_NAME_CONFIG);

    }


    @Override

    public T deserialize(String topic, byte[] data) {

        if (data == null) {

            return null;

        }

        try {

            return objectMapper.readValue(data, className);

        } catch (Exception e) {

            throw new SerializationException(e);

        }

    }

}
```

```
}
```

2.2) Tạo Invoice POJO

```
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
@JsonInclude(JsonInclude.Include.NON_NULL)
public class Invoice {

    private String invoiceNumber;

    private String storeId;

    private long created;

    private double totalAmount;

    private boolean valid;

}
```

2.3) Consumer

Tạo consumer và produce, sau đó consume message và produce đến topic tương ứng. Để đơn giản, chúng ta sử dụng luôn field `valid` để check message:

```
@Slf4j
public class ValidatorConsumer {
```



```
@SuppressWarnings("InfiniteLoopStatement")

public static void main(String[] args) {

    final var consumerProps = new Properties();

    consumerProps.put(ConsumerConfig.CLIENT_ID_CONFIG, "validation-
consumer");

    consumerProps.put(ConsumerConfig.BootstrapServers_CONFIG,
"localhost:9092,localhost:9093");

    consumerProps.put(ConsumerConfig.KeyDeserializer_CONFIG,
StringDeserializer.class);

    consumerProps.put(ConsumerConfig.ValueDeserializer_CONFIG,
JsonDeserializer.class);

    consumerProps.put(JsonDeserializer.VALUE_CLASS_NAME_CONFIG,
Invoice.class);

    final var consumer = new KafkaConsumer<String,
Invoice>(consumerProps);

    consumer.subscribe(List.of("invoice-topic"));

    final var producerProps = new Properties();

    producerProps.put(ProducerConfig.CLIENT_ID_CONFIG, "validation-
producer");

    producerProps.put(ProducerConfig.BootstrapServers_CONFIG,
"localhost:9092,localhost:9093");

    producerProps.put(ProducerConfig.KeySerializer_CONFIG,
StringSerializer.class);

    producerProps.put(ProducerConfig.ValueSerializer_CONFIG,
JsonSerializer.class);
```

```

final var producer = new KafkaProducer<>(producerProps);

while (true) {
    final var records = consumer.poll(Duration.ofMillis(100));
    records.forEach(r -> {
        if (!r.value().isValid()) {
            producer.send(new ProducerRecord<>("invalid-invoice-
topic", r.value().getStoreId(), r.value()));

            log.info("Invalid record - {}",
r.value().getInvoiceNumber());

            return;
        }

        producer.send(new ProducerRecord<>("valid-invoice-topic",
r.value().getStoreId(), r.value()));

        log.info("Valid record - {}",
r.value().getInvoiceNumber());
    });
}
}
}

```

2.4) Producer

Cuối cùng, tạo producer để produce message đến **invoice-topic**:

```

public class InvoiceProducer {

```

```

public static void main(String[] args) {

    final var random = new Random();

    final var props = new Properties();

    props.put(ProducerConfig.CLIENT_ID_CONFIG, "producer");

    props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
"localhost:9092,localhost:9093");

    props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
StringSerializer.class);

    props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
JsonSerializer.class);

    try (var producer = new KafkaProducer<String, Invoice>(props)) {

        IntStream.range(0, 1000)

            .parallel()

            .forEach(i -> {

                final var invoice = Invoice.builder()

                    .invoiceNumber(String.format("%05d", i))

                    .storeId(i % 5 + "")

                    .created(System.currentTimeMillis())

                    .valid(random.nextBoolean())

                    .build();

                producer.send(new ProducerRecord<>("invoice-
topic", invoice));

            });

    }

}

```

```
}
```

2.5) Tạo topic

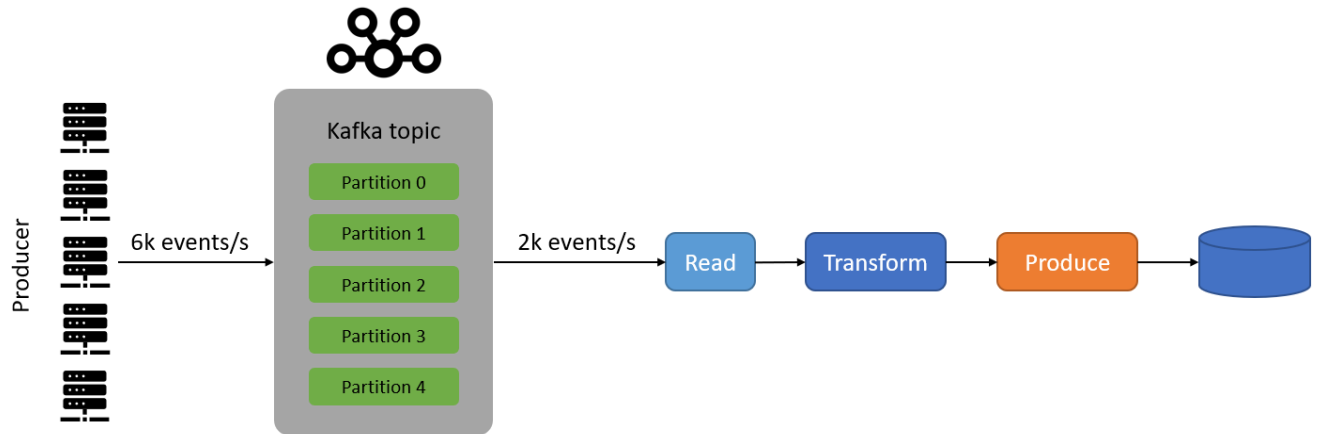
Tạo **invoice-topic** với 2 partitions:

```
$ kafka-topics.sh \  
  --bootstrap-server localhost:9092,localhost:9093 \  
  --partitions 2 \  
  --replication-factor 3 \  
  --topic invoice-topic \  
  --create
```

Sau đó start **Producer** và **Consumer** và tận hưởng thành quả thôi.

3) Scale Kafka consumer với **group.id**

Thực tế, các bài toán không chỉ đơn thuần check choác như trên mà phức tạp hơn nhiều. Như vậy, trong trường hợp có rất nhiều **producer** nhưng chỉ có một **consumer** thì không ổn. Lúc này **consumer** trở thành bottle-neck của hệ thống, khiến cho application không còn tính chất **real-time**.



Trong trường hợp này, cách thông dụng nhất là scale **consumer** để handle đồng thời nhiều message hơn bằng cách sử dụng **consumer group** đã giới thiệu ở bài trước.

```
consumerProps.put(ConsumerConfig.GROUP_ID_CONFIG, "validator-group");
```

Sau khi thêm properties, start 2 **consumer** và tiến hành produce message chúng ta sẽ thấy các message được route đến cả 2 **consumer**.

Nếu thêm 1 **consumer** nữa, lúc này có 2 partition và 3 consumer, tuy nhiên chỉ 2 consumer nhận được message. Nếu chưa hiểu nguyên nhân có thể đọc lại phần trước về **consumer group**.

[Download source code here.](#)